

Продвинутый NGO

события, пул объектов и сетевое лобби

Завершение Блока 1 — полный арсенал NGO перед миграцией на FishNet

Targeted RPC

Unreliable RPC

Object Pooling

NetworkList

Инвентарь

Лобби

Scene Management

Карта занятия

1

Продвинутые RPC-паттерны

Targeted RPC · NV vs RPC матрица · Unreliable RPC

2

Object Pooling

INetworkPrefabInstanceHandler · серверный и клиентский пул

3

Сетевой инвентарь

NetworkList · подбор/выброс с серверной валидацией

4

Лобби

Подключения · готовность · NetworkManager.SceneManager

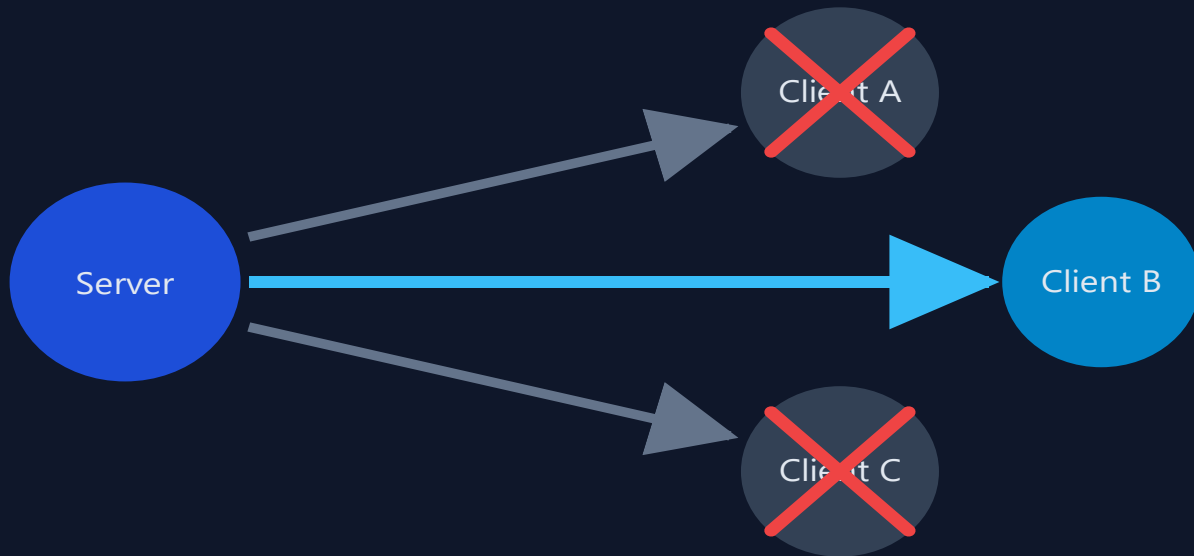
5

Wrap-up Блока 1 → FishNet

Итог · ограничения NGO · мост к Лекции 5

Targeted RPC — сообщение одному клиенту

Targeted ClientRpc



Targeted RPC — как это работает



ClientRpcParams

TargetClientIds — массив ulong-получателей. Пусто = broadcast всем.



Когда нужен

Уведомления, приватные данные, торговля между двумя игроками.



Экономия трафика

16 игроков, 1 получатель — 15 клиентов не тратят полосу на обработку.

```
ClientRpcParams rpc = new() { Send = new() { TargetClientIds = new ulong[] { clientId } } };
```

NetworkVariable vs RPC — матрица решений

NetworkVariable vs RPC

Question 1: needed for late join? Question 2: event or persistent state?

One-time event

Persistent state

Both

NetworkVariable

RPC

Skip or Local

Not needed for late join
Needed for late join

NetworkVariable vs RPC — правила

NetworkVariable

КОГДА: Нужно при late-join

- HP, счёт, состояние двери
- Текущее оружие игрока
- Открыта / закрыта ли дверь

RPC

КОГДА: Одноразовое событие

- Звук выстрела, анимация подбора
- Уведомление игроку
- Чат-сообщение

⚡ Оба: смерть =

`NetworkVariable<bool> IsAlive + ClientRpc PlayDeathEffect()`

Unreliable RPC — экономия на некритичных событиях

Reliable (по умолчанию)

- Гарантия доставки + ACK-подтверждение
- 60 событий/сек = 60 лишних ACK-пакетов
- Обязателен: выстрел, подбор, смерть

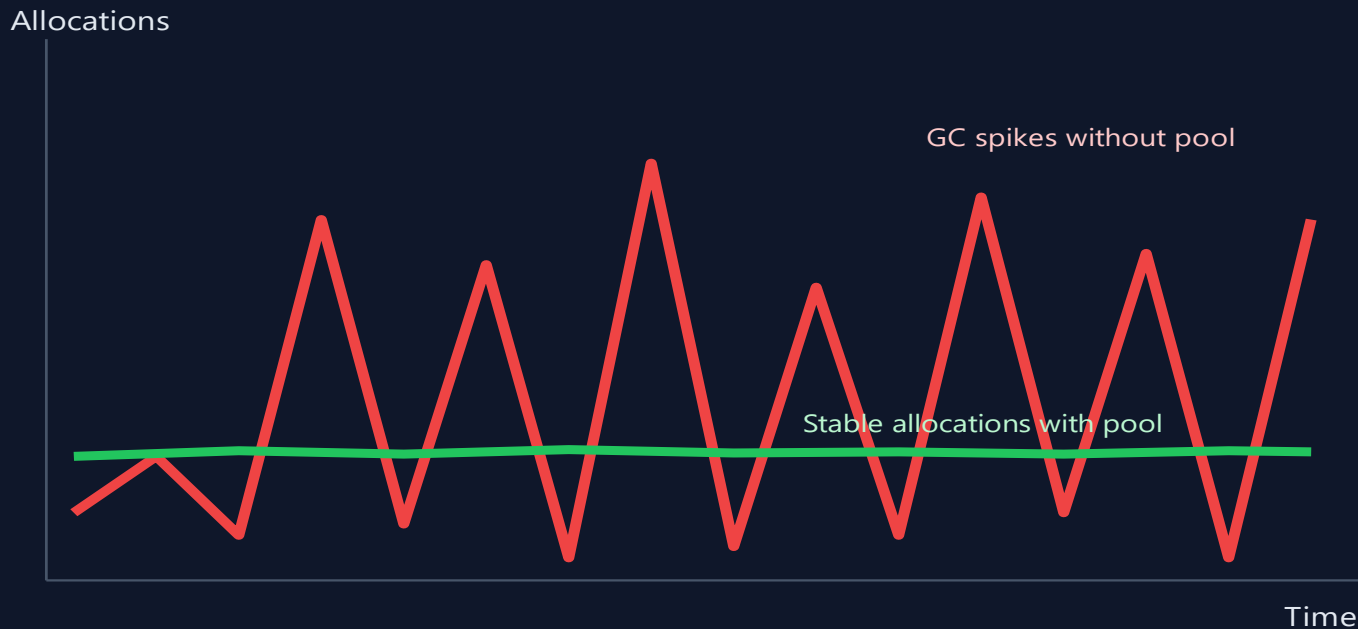
Unreliable

- Без подтверждения — быстрее, меньше трафика
- Потеря одного пакета несущественна
- Для: следы пуль, шаги, частицы, HUD

```
[ServerRpc(Delivery = RpcDelivery.Unreliable)] void FootstepSoundServerRpc(Vector3 pos) { }
```

Object Pooling — почему Instantiate не масштабируется

Spawn/Despawn vs Pooling



16 игроков × 2 выстр/сек × 3 сек = 96 аллокаций/сек → GC spikes → фризы

Object Pooling — решение

БЕЗ ПУЛА

No Pool: create to fly to destroy



Instantiate → Spawn → Despawn → Destroy
Каждый раз: аллокация + GC

С ПУЛОМ

Pool: activate to fly to deactivate to return



Активировать → использовать → деактивировать
Ноль аллокаций в рантайме

INetworkPrefabInstanceHandler — серверный пул

Instantiate()

Берём из очереди или создаём новый (если пул пуст).

Destroy()

Деактивируем и возвращаем в очередь — без реального удаления.

Pre-warming

Создаём все объекты при старте. Ленивое создание = снова GC.

Формула размера

$\text{maxPlayers} \times \text{fireRate} \times \text{maxLifetime}$. Пример: $16 \times 2 \times 3 = 96$.

```
NetworkManager.Singleton.PrefabHandler.AddHandler(prefab, new ProjectilePoolHandler(prefab, 96));
```

Два независимых пула в проекте

Серверный пул

`INetworkPrefabInstanceHandler`

- Управляет `NetworkObject`-снарядами
- Подменяет `Instantiate/Destroy` для `NGO`
- Регистрируется через `PrefabHandler`
- Pre-warming при старте сервера

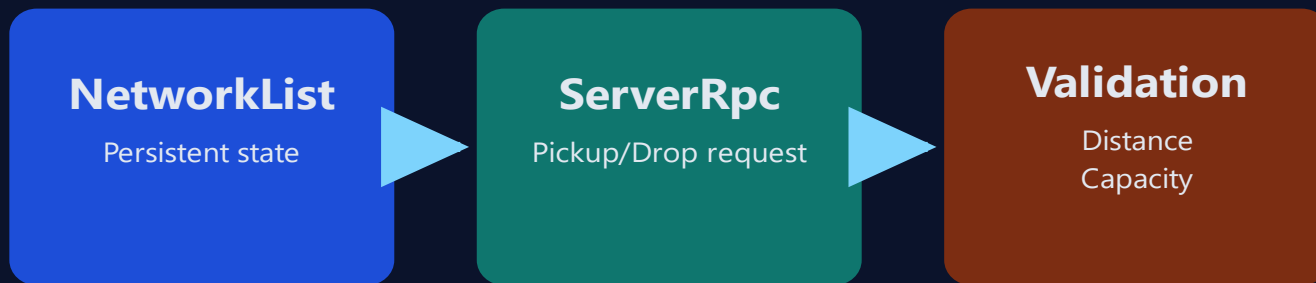
Клиентский пул

`ObjectPool<ParticleSystem>`

- Управляет локальными VFX-эффектами
- Обычный Unity `ObjectPool<T>`
- Запускается из `ClientRpc`
- Возврат через корутину (`WaitForSeconds`)

Архитектура сетевого инвентаря

Network Inventory Architecture



NetworkList vs NetworkVariable — почему важна дельта

NetworkVariable<массив>

- Отправляет ВЕСЬ массив при любом изменении
- 50 слотов = 50 элементов по сети
- Растёт с размером инвентаря

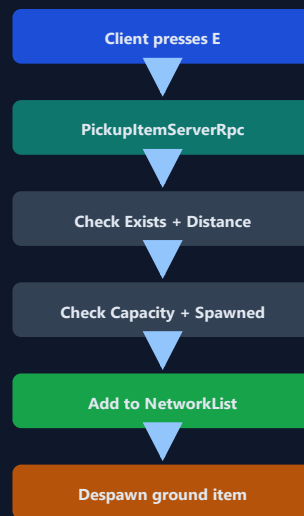
NetworkList<T>

- Отправляет ДЕЛЬТУ: «добавлен элемент в #3»
- 1 изменение = 1 маленький пакет
- Не зависит от размера списка

InventorySlot : INetworkSerializable, IEquatable<InventorySlot> → порядок полей обязателен!

Полный цикл подбора предмета

Pickup Item Flow



Exists (TryGet)

Distance $\leq 2.5f$

Capacity < Max

IsSpawned

4 guard-клоза — принцип тот же, что в стрельбе

1

Exists

Предмет существует в сети?

```
if (!itemRef.TryGet(out NetworkObject itemObj)) return;
```

2

Distance

Расстояние допустимо (античит)?

```
if (dist > 2.5f) return;
```

3

Capacity

Есть место в инвентаре?

```
if (Inventory.Count >= MaxSlots) return;
```

4

Not Taken

Предмет ещё не подобран другим?

```
if (!itemObj.IsSpawned) return;
```

Сравните с ShootServerRpc: Owner / Alive / Cooldown / Direction — принцип один.

Обработка подключений и отключений

OnClientConnectedCallback

Клиент подключился → сохраняем данные, уведомляем остальных

OnClientDisconnectCallback

Клиент отключился → чистим данные, уведомляем остальных

OnNetworkDespawn()

ОБЯЗАТЕЛЬНО отписаться! Иначе — утечка памяти и двойные вызовы

Грaciousное отключение → NetworkManager.Shutdown() → мгновенно

Потеря связи → timeout 10–30 сек → затем OnClientDisconnect

Готовность игроков — паттерн Lobby → Ready → Game

Lobby to Match Flow



```
if (readyStates.Count >= 2 && readyStates.Values.All(r => r)) → StartGame()
```

Scene Management — локальный vs сетевой

`SceneManager.LoadScene()`

- Загружает только на текущей машине
- Другие клиенты не затронуты
- Рассинхронизация состояния

`NetworkManager.SceneManager.LoadScene()`

- Загружает для ВСЕХ клиентов синхронно
- Только сервер вызывает!
- Основа игрового цикла Лобби → Игра → Результаты

Паттерн: постоянная сцена + загружаемая

Bootstrap (NetworkManager, UI, Audio) — постоянная · Arena / Lobby — меняется

Полный арсенал NGO за 4 лекции

Л1

Архитектура NGO

NetworkManager · NetworkObject · RPC · NetworkVariable · Топологии

Л2

Сериализация и трафик

FastBuffer · INetworkSerializable · Квантизация · Packed int

Л3

Transform, Ownership, Стрельба

NetworkTransform · Jitter Buffer · ChangeOwnership · ShootServerRpc

Л4

Продвинутые паттерны

Targeted/Unreliable RPC · Object Pooling · NetworkList · Лобби · Scene Mgmt

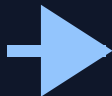
Лекция 5: миграция на FishNet — всё то же самое + Prediction + Observers

Ограничения NGO — зачем переходить на FishNet

NGO Foundation to FishNet Upgrade

NGO

- RPC + NetworkVariable
- Good for prototype
- Limited observers
- No built-in prediction



FishNet

- Better scalability
- Observer conditions (AOI)
- Prediction-ready stack
- Lower alloc pressure

✗ Нет Prediction — задержка при пинге 150 мс

✗ Примитивные Observers — все объекты всем

✗ Нет Host Migration — хост вышел = игра упала

✗ 32+ клиентов × 500+ объектов = проблемы